

South Barnegat Bay Onshore Wind Model Prediction with the Use of Long Short-Term Memory Neural Networks

Krupam Patel

36740@ocvts.org

Marine Academy of Technology and Environmental Science

Sophomore Research Paper 2025-2026

Abstract:

Persistent South and Western onshore winds initiate the summertime event known as upwelling. Upwelling in small, localized areas like Barnegat Bay, NJ, has a significant impact on bay temperature, creating a large land-sea temperature difference. This difference can lead to harsh and fast onshore breezes that can “swamp” small watercraft. This study uses a Long Short-Term Memory (LSTM); a deep learning neural network, to predict these large gusts, along with Naive and Linear Regression algorithms to verify its effectiveness. To utilize these models, thirteen variables were collected on an hourly basis for June-September. Once the models were created and trained, the mean absolute error was calculated for each of the models as a comparison. Shockingly, it seemed that Linear Regressions and Naive models performed marginally better than the LSTM, which had collapsed to predicting a value close to the mean in almost all tests. Accurate wind speeds and direction were still predicted, as the hypothesis says, just with different models. With the rarity of upwelling events and their spontaneity, it was challenging, yet the models were able to predict them.

Introduction:

Upwelling is a crucial part of coastal ecosystems. In this process, the seafloor's deep, nutrient-rich, cold water is mixed with the surface water, cooling the entire body of water, among other effects. At sea, persistent South, Southeast, or West winds (damp onshore winds) push warm water off the surface of bays, estuaries, and coastal waters, initiating the upwelling process (For Barnegat Bay, NJ). West winds create quick bursts of upwelling, while South and Southeast winds create a steadier upwelling over time. These winds have this power due to the effect of Ekman transport, where winds drive the movement of surface ocean water in conjunction with the Coriolis effect: “the pattern of deflection taken by objects not firmly connected to the ground as they travel long distances around Earth...the impact of the Coriolis Effect helps define regular

wind patterns around the globe” (Evers 2025). This large-scale effect can be traced back to the Bermuda High: a semi-permanent high-pressure system directing storms, winds, and more up towards the east coast. All of these weather patterns allow upwelling with the damp wind directions; however, this cannot be said for the drier directions of East and Northeast. But on a local scale, in bays with upwelling due to the Bermuda High winds, a larger temperature gradient between the water and land can be seen, especially because of local man-made structures that heat up faster; (e.g. a parking lot). The air temperature could be 30° C, while the body of water could reasonably be 11 - 12° C during the summer. These drastic differences lead to giant gusts of onshore winds that originate from the coast, resulting from the rising warm air from land and denser, colder air from the ocean. High tide brings in the cooler water from the upwelling, and low tide carries it away from the bay. These winds flow over the bay and, through Ekman transport, create high wave activity in the bay that can “swamp” small watercraft, from canoes to jet skis.

Predicting these gusts with high accuracy within short time frames can help anyone on the water prepare and stay safe. Traditional weather forecasting models, at these larger scales, often lack the ability to capture the rapid, small-scale changes that affect localized areas like the southern part of Barnegat Bay, which will be studied in this report. These models may generalize wind patterns over broad areas and periods, missing key factors such as sudden upwelling events or sharp land-sea temperature gradients that can form in a matter of hours.

One approach to predict these sudden onshore wind gusts is a Long Short-Term Memory Model (LSTM), to analyze time-series weather data and predict the next hour of onshore wind levels in Barnegat Bay. LSTM networks are a specialized form of Recurrent Neural Network (RNN), a “deep learning model that is trained to process and convert a sequential data input into

a specific sequential data output.” (AWS 2025). LSTMs store memory differently compared to other neural networks, which allows them to remember, update, or “forget” data through different logic gates. These gates regulate the flow of information, allowing for the model not to be overwhelmed with data and keeping the context of the situation, a major disadvantage in typical RNNs. An LSTM could learn to recognize weather patterns and upwelling events to predict wind speeds and direction. Utilizing an LSTM could help predict strong onshore winds and protect people out on the water with the development of a warning system. It is hypothesized that an LSTM neural network will be able to accurately predict onshore wind strength and direction on a one-hour-ahead basis, using actual wind speed and direction values taken from weather stations and buoys on the bay and ocean side for the predicted hours.

Methodology:

- A Long Short-Term Memory (LSTM) neural network will be built and trained on collected weather data to forecast next-hour wind speeds and directions when onshore conditions occur.

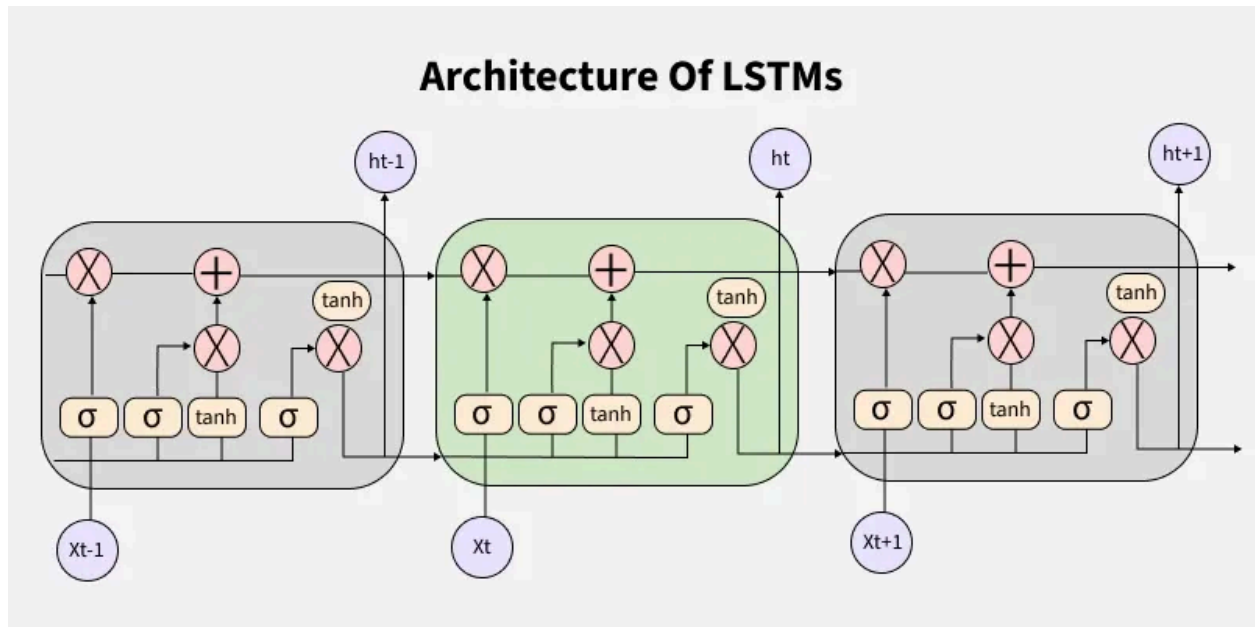


Figure 2: LSTM Cell Diagram from [geeksforgeeks.org](https://www.geeksforgeeks.org/lstm-cell-diagram/)

- The Naive and Linear Regression algorithms will be tested along with the LSTM to verify the necessity of the deep learning model. There will be a model for every variable predicted. The most accurate of the three will be used in the final program.
 - The Mean Absolute Error (MAE) will be calculated and noted for all the predictions compared to the real hour measurements from the Stafford Weather Station, NJ, and then compared to find the most accurate prediction
 - MAE values show the average value of error.
 - Naive Algorithm: This algorithm assumes that the value presented/predicted before is correct and uses this value for its prediction. It also assumes that values change slowly over time.

- Linear Regression Algorithm: This algorithm assumes that there is a linear relation between the input and output as the best-fit line for the prediction. In multiple variable situations like this, it uses weighted sums of the current values.
- Variables to be predicted
 - Several variables will be predicted by the models:
 - Wind Speed (mph)
 - Wind Gust Speed (mph)
 - Wind Direction
 - If it is an onshore wind
 - If it is an upwelling event
- Training data collection occurred every hour, starting June 1, 2025. The collection was conducted during the summer months, concluding on September 24, 2025 (Due to the shutdown of the NJDEP MB_01 Buoy).
 - Due to this shutdown, the data was shrunk to just the summer to avoid biasing the data.
 - Variables to be noted:
 - Wind Speed in mph (Stafford Weather Station)
 - Wind Gusting in mph (Stafford Weather Station)
 - Wind direction labels (N, NNE, etc) (Stafford Weather Station)
 - Humidity Percentage (Stafford Weather Station)
 - Precipitation in inches (Stafford Weather Station)
 - Atmospheric Pressure in inches (Stafford Weather Station)

- Mainland Air temperature in fahrenheit (Stafford Weather Station)
- LBI Air temperature in fahrenheit (Surf City Yacht Club Weather Station)
- Surface Bay water temperature in celsius (NJDEP MB_01 Buoy)
- Salinity in ppt (NJDEP MB_01 Buoy)
- Surface Ocean water temperature in celsius (NDBC Station 44091)
- (All data in fahrenheit will be converted to celsius)

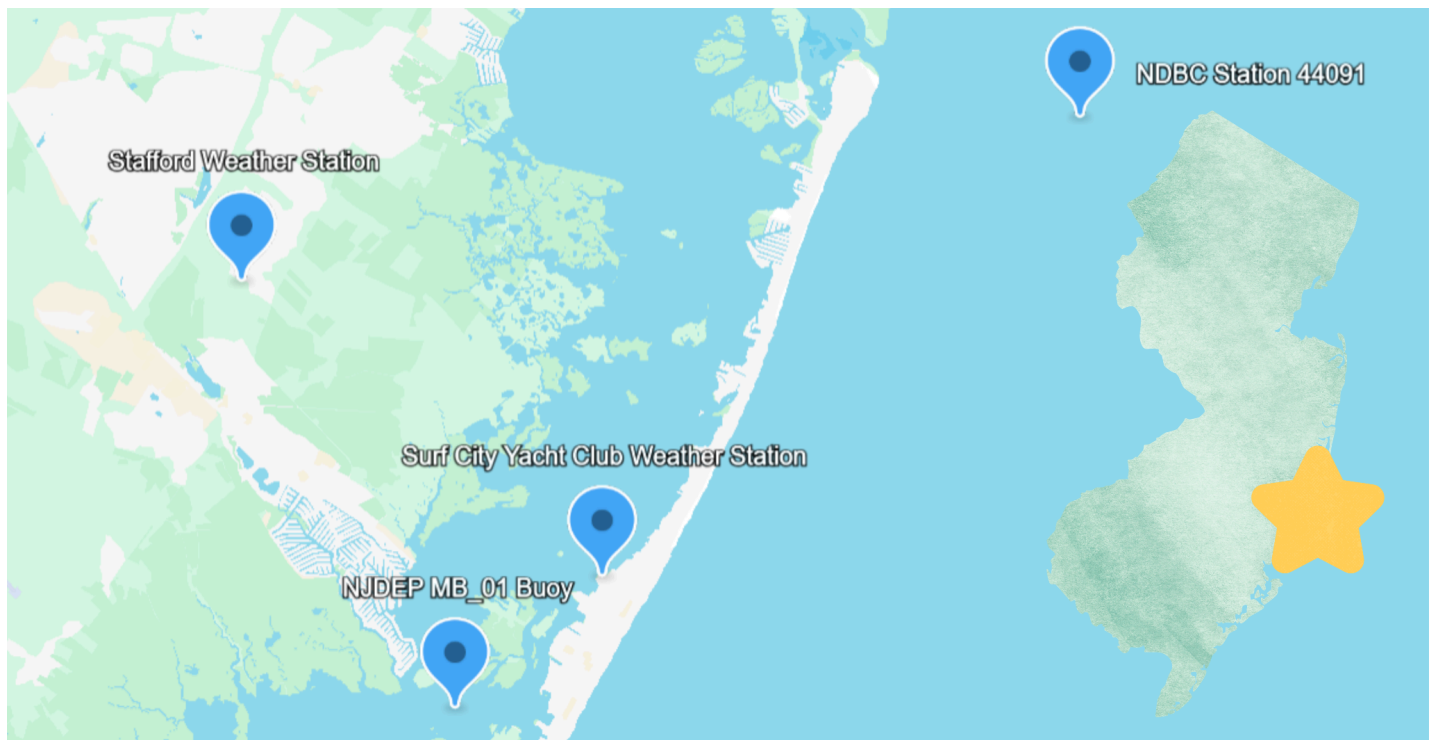


Figure 2: Weather Stations and Buoys in a triangular active monitoring setup (LBI, NJ).

- LSTM Input and Prediction
 - The LSTM will be trained with the collected data before being able to pull the last 24 hours' worth of data collected from the above-mentioned weather stations and buoys
- Naive and Linear Regression Input and Prediction

- The Naive algorithm will only need to pull the last data point to predict.
- The Linear Regression was trained with the same testing data (like the LSTM) and will only need a single hour of data to predict the next hour.
- Weather Stations and Buoys:
 - Stafford Weather Station: Provides most variables and is where the predictions are all trained for.
 - NJDEP MB_01 Buoy: Provided bay water temperature and salinity that is used to contrast the land-bay relationship.
 - Surf City Yacht Club Weather Station: Contrasted the mainland air temperature with data from the island for a land gradient.
 - NDBC Station 44091: Ocean Buoy provided ocean temperatures for comparisons with bay water for determining upwelling.
 - Data was formatted with the date, time (hours 0-23), direction as text, and all values recorded as floats.
 - Data was collected manually by reviewing specific links to the sensors and weather stations.
- Programming Setup:
 - All data was collected using google sheets before being converted to a csv file for the program to use.
 - The models and data analyzing was coded in a single Visual Studio Code Project in Python.
 - Progress and backup were monitored by the project's GitHub page:
 - <https://github.com/Krupamc/Research-2026-LSTM>

- Data Cleaning and Formatting (Script):
 - This script uses the datetime, os, pandas, and numpy libraries.
 - Steps for this script:
 - Import the raw data as a csv dataframe with the Pandas library
 - Utilizing a Python map, the wind directions were converted into “bins” for the models to more accurately predict with a range of 0-15.
 - Convert all Fahrenheit points to Celsius.
 - Round all points to either one or two decimals after the point.
 - Check if the current hour wind direction was onshore (South through Northeast), and if so, increment the onshore wind flag column for that row to one or yes.
 - If all conditions are filled, the upwelling_flag column for that row is incremented to one or yes.
 - Check if the wind direction is a damp upwelling wind (SE, SSE, S, W)
 - The rolling mean of the last 48 hours of ocean temperature is subtracted from the rolling mean of the last 6 hours of ocean temperature.
 - If this number is less or equal to than a defined threshold (-3.0), and is sustained for 6 hours it is considered upwelling.
 - Save this csv as the “clean” csv.
- The LSTM Model:

- This script uses the StandardScaler (from Sklearn.preprocessing), mean_absolute_error (from Sklearn.metrics), keras (from tensorflow), joblib, datetime, os, pandas, and numpy libraries.
- This script was designed to take the previous 24 hours of weather data to predict the next hour's value. There was a separate model for each data point being predicted.
- Steps for this script:
 - Open the “clean” csv as a pandas dataframe and convert the Date and Hour columns into one Date column as a Datetime object from pandas.
 - From there, a month column was created with a ‘summer_mask’ to restrict data to June through September.
 - Turn all columns into Python variables.
 - Save all input variables into one numpy array.
 - Save all output variables into another numpy array and reshape it to a 1D array.
 - Save the length of the training data to a variable (90% train and 10% test)
 - Use StandardScale to scale the data or standardize the input and output features by transforming them to have a mean of 0 and a standard deviation of 1 for the model to understand.
 - Create the X_train and y_train variables.
 - X_train = input (Independent Variable)
 - y_train = output (Dependent Variable)

- Create a sliding window over the last 24 hours of data and append the train variables.
- Create the X_test and y_test variables. (Same as train, but to rebuild the data after prediction)
- Create a sliding window over the last 24 hours of data and append the test variables.
- Convert X_train and y_train from lists to arrays. (X_train = 3D array, y_train = 1D array)
- Recreate the window for X_train and y_train.
- Build the model sequentially:
 - Layer Zero: Set the shaping of the model in preparation for the data
 - Layer One: The LSTM learns 32 time-dependent features (With its 32 cells) from each 24-hour window.
 - Layer Two: A Dense layer refines these 32 features (With its 32 cells) into more abstract patterns
 - Layer Three: A Dense layer that takes these 32 numbers into a single predicted outcome.
 - Many fine-tunings were done to get the highest performance of the LSTM, including;
 - Epoch Number - The number of passes through a dataset.

- Validation Split - Splitting Factor of how much of the data is set aside for testing to avoid over-fitting or making the model inefficient in predicting new data.
 - Shuffling - Whether or not the data is shuffled around to prevent over-fitting.
 - Unscale the data to get readable results.
 - Calculate the MAE and save the results to a file.
 - Save the trained model to a file.
- Linear Regression:
 - This script uses the StandardScaler (from Sklearn.preprocessing), LinearRegression (from Sklearn.linear_model), mean_absolute_error (from Sklearn.metrics), keras (from tensorflow), joblib, datetime, os, pandas, and numpy libraries.
 - This script was designed to use a line of best fit and a weighing system to predict the next hour's value. There was a separate model for each data point being predicted.
 - Steps for this script:
 - Open the "clean" csv as a pandas dataframe and convert the Date and Hour columns into one Date column as a Datetime object from pandas.
 - From there, a month column was created with a 'summer_mask' to restrict data to only summer months.
 - Turn all columns into Python variables.
 - Save all input variables into one numpy array.

- Save all output variables into another numpy array and reshape it to a 1D array.
 - Save the length of the training data to a variable (90% train and 10% test)
 - Use StandardScale to scale the input data.
 - Create the X_train and y_train variables.
 - Convert X_train and y_train from lists to arrays. (X_train = 3D array, y_train = 1D array)
 - Create the model and fit the data.
 - Predict the data, round, and prevent negatives.
 - Save the results.
 - Calculate the MAE and save the trained model to a file.
- Naive:
 - This script uses the mean_absolute_error (from Sklearn.metrics), datetime, os, pandas, and numpy libraries.
 - This script was designed to predict the last value using Naive. There was a separate model for each data point being predicted.
 - Steps for this script:
 - Create a variable for the model.
 - Tell the model which algorithm to use (Naive) and the rule.
 - Predict the value, round, and prevent negatives.
 - Save the results.
 - Calculate the MAE.
- Final Script:

- At the end, all of the accurate models were compiled and run in a single script, which can be seen along with all of the other models in the GitHub repository: <https://github.com/Krupamc/Research-2026-LSTM>. Instruction, a quick overview, and archived models can be found here.

Results:

Figure 3: MAE values of every model tested, with the chosen models highlighted.

Model MAE's	Wind Speed	Wind Gusting	Wind Direction	Onshore	Upwelling
Linear MAE	0.56	0.96	3.92	0.55	Collapsed
Naive MAE	1.13	1.86	1.17	0.17	Collapsed
LSTM MAE	Collapsed	Collapsed	Collapsed	Collapsed	Collapsed
Onshore from direction MAE	-	-	-	0.13	-

Discussion:

The MAE values highlight the strengths and weaknesses of each model. Linear Regression performed best when it comes to predicting values that have a relationship between the input and output. The Naive model performs relatively better with the slower changing variables like wind direction and rare occurrences of upwelling. The Linear Regression model for wind speed manages to achieve a MAE of 0.56. As expected with more volatile winds, the MAE is larger, at 0.96. This model is able to almost reduce the MAE by half compared to the Naive model. All of this changes when predicting wind direction, Naive generates a MAE of

1.17 compared to Linear Regression's 3.97. However, with the way that the directions were coded, each of the 16 direction 'bins' represented 22.5°. Therefore, this 1.17 MAE is roughly 26° off. Though not as accurate as expected, this direction prediction is accurate enough for general people out on the water. From our direction prediction, we check whether each value is onshore (NNE through S) and obtain an MAE of 0.13. (approximately 87% accuracy) The rarity of upwelling events made it harder for not just the Naive model but the rest as well. This unbalanced data set led to the collapse of all models. Model collapse is defined as "a degenerative process affecting generations of learned generative models, in which the data they generate end up polluting the training set of the next generation" (Shumailov 2024). On that note, LSTMs can, in principle, handle very large datasets but their performance in this project degraded due to being trained on highly imbalanced data in the case of upwelling. This is why in field application, these more complex models are run in-accompanying simpler models to regulate the necessity. These bigger models perform very effectively when they can but often can be an unnecessary cost of materials, processing power, and time when a simpler model can do the same job quicker and more efficiently.

Conclusion:

Though the predictions may not be made with the same model, one thing is certain: this program can still predict wind speeds, wind gust speeds, and wind direction with accuracy. This project aimed to determine whether this program would be helpful for people on the water, and the results suggest that it is. The prediction for the wind direction has a greater margin of error, but to someone on the water, just knowing the general direction matters more than an exact degree. These tools, wrapped into a simple

program warning system, can provide the nitty-gritty detail generalized national weather predictions might miss. In the future, exploring even more models and designs compared to these simpler models could bring even more value to this small little program, especially if it is deployed on a website for any boater or paddler to access for real-time guidance.

References:

- AWS. (2025). *What is RNN? - Recurrent Neural Networks Explained - AWS*. Amazon Web Services, Inc. <https://aws.amazon.com/what-is/recurrent-neural-network/>
- Brockwell, P., Davis, R., Gardner, G., Harvey, A., Phillips, G., West, M., Harrison, J., Ripley, B., & Meyer, D. (2002). Time Series: A Biostatistical Introduction. *Naive Time Series Forecasting Methods*, 2(2).
<https://journal.r-project.org/articles/RN-2002-008/RN-2002-008.pdf>
- Dabulevičienė, T., Nesteckytė, L., & Kelpšaitė-Rimkienė, L. (2024). Assessing the effect of coastal upwelling on the air temperature at the south-eastern coast of the Baltic Sea. *Oceanologia*, 66(2), 394–404. <https://doi.org/10.1016/j.oceano.2024.03.002>
- GeeksforGeeks. (2025, December 23). *Deep Learning | Introduction to Long Short Term Memory*. GeeksforGeeks.
<https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/>
- GeeksforGeeks. (2026, February 23). *Linear Regression in Machine learning*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>
- Murphy, J. (2020, August 13). *What is the Bermuda high, and how does it affect tropical systems?* Firstcoastnews.com; WTLV-TV.

<https://www.firstcoastnews.com/article/weather/accuweather/what-is-the-bermuda-high-defined-definition/507-b68965cb-a1a5-4ac8-8843-ee51e2273b28>

National Geographic. (2022, September 27). *The Coriolis Effect: Earth's Rotation and Its Effect on Weather* | National Geographic Society. Education.nationalgeographic.org.

<https://education.nationalgeographic.org/resource/coriolis-effect/>

Quan, J. (2021, March 22). *What is Coastal Upwelling and Why is it Important?* Coastal and Marine Sciences Institute. <https://marinescience.ucdavis.edu/blog/upwelling>

Redmon, M. (2024, March 28). *How Accurate Are Weather Forecasts? 5 Reasons For Inaccuracy*. WeatherFlow-Tempest, Inc.

<https://tempest.earth/resources/how-accurate-are-weather-forecasts/>

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., & Gal, Y. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631(8022), 755–759.

<https://doi.org/10.1038/s41586-024-07566-y>

Time series forecasting. (2024). *Time series forecasting* | TensorFlow Core. TensorFlow.

https://www.tensorflow.org/tutorials/structured_data/time_series

Wenegrat, J. O., & Thomas, L. N. (2017). Ekman Transport in Balanced Currents with Curvature. *Journal of Physical Oceanography*, 47(5), 1189–1203.

<https://doi.org/10.1175/jpo-d-16-0239.1>